Úloha 09/03

Psí salon

1 Zadání

> *Do psího salonu přichází zákazníci v intervalech exponenciálně se středem 10 minut. Salon má jednu linku obsluhy, obsluha (stříhání) trvá exponenciálně se středem 15 minut. Pokud některý zákazník čeká ve frontě alespoň 1 hodinu, všichni čekající zákazníci salon opustí a salon na přesně 3 hodiny přestane přijímat nové zákazníky.*

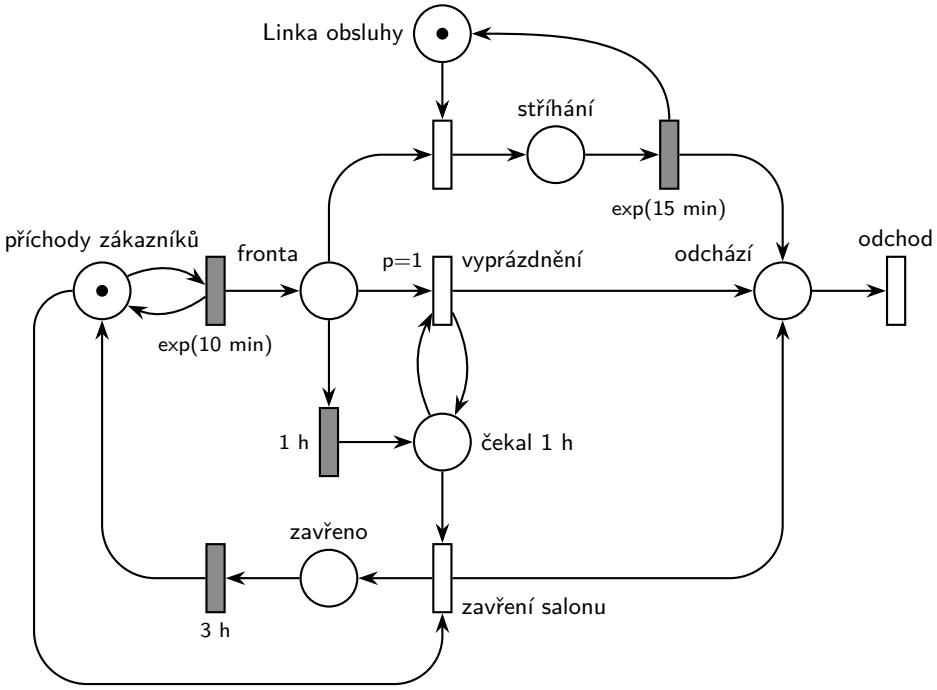
Všimni si: obsluha (15 min) je pomalejší než příchody (10 min), takže bez pravidla s jednou hodinou by fronta rostla do nekonečna. Právě proto zadání dává smysl.

Podobné úlohy: Přerušení příchodů při plné frontě, Každá 100. transakce vyprázdní frontu, Netrpělivé požadavky.

2 Nápověda

- **Příchody** modeluje místo *příchody zákazníků* s jednou značkou ve smyčce s časovaným přechodem `exp(10 min)`; zákazník čeká v místě *fronta*.
- **Jedna linka obsluhy** je místo *Linka obsluhy* s jednou značkou; obsazení ji bere, časovaný přechod `exp(15 min)` ji vrací (uvolnění je s ním sloučené). Obsloužený zákazník jde do místa *odchází*.
- **Hodina čekání:** časovaný přechod `1 h` má vstup jen z *fronty*. Podle slajdu 138 má každý čekající vlastní odpočet, takže se přechod provede, jakmile některý zákazník čeká ve frontě hodinu. Kdo se dřív dostane k lince, tomu obsazení vezme značku a jeho odpočet zanikne (zaniká nejstarší odpočet, což odpovídá frontě FIFO, viz Jak síť funguje). Během zavření je fronta prázdná a nikdo nepřichází, takže žádný odpočet neběží a další zavření se nevyhlásí.
- Přechod `1 h` vezme z fronty zákazníka, který čekal hodinu. Ten se stane značkou v místě *čekal 1 h*. Odchází ze systému až přechodem *zavření salonu*.
- **Vyprázdnění fronty:** okamžitý přechod s `p=1` opakovaně bere zákazníky z *fronty* do místa *odchází* a vrací značku *čekal 1 h* zpět (smyčka), dokud fronta není prázdná. Priorita `p=1` zajistí, že vyprázdnění proběhne dřív než *zavření salonu* (p=0) a dřív než případné obsazení linky.
- **Zavření salonu** odebere značku z místa *příchody zákazníků*, takže probíhající odpočet `exp(10 min)` zanikne a nikdo nepřichází; zákazníka z místa *čekal 1 h* pošle do místa *odchází*. Po `3 h` se značka příchodů vrátí.
- Zákazník, který je právě obsluhován, obsluhu normálně dokončí.
- Všichni zákazníci **opouští systém** přes místo *odchází* okamžitým přechodem *odchod* bez výstupních hran.

3 Petriho síť



Obrázek 1. Petriho síť – psí salon

4 Časté chyby

- Zastavení příchodů odebráním značky z *fronty* místo z místa *příchody zákazníků*. Generátor musí přijít o svou značku, jinak příchody pokračují.
- Vyprázdnění fronty jedním přechodem s váhou hrany. Váha bere přesný počet značek, ne „všechny“. Na „všechny“ je potřeba smyčka s prioritou.
- Zapomenutá smyčka *čekal 1 h* → přechod → *čekal 1 h*: pak se odebere jen jeden zákazník.
- Hodinový odpočet s řídicí značkou (jeden odpočet pro celou frontu). Ten měří, jak dlouho je fronta nepřetržitě neprázdná, ne kolik čeká konkrétní zákazník. Zákazníci se ve frontě střídají, takže zavření by přišlo, i když nikdo hodinu nečekal.

5 SIMLIB

Model je izomorfní k Petriho síti výše: každé místo, přechod a značka mají svůj protějšek v kódu.

```
// Psí salon - model v SIMLIB/C++
#include "simlib.h"

Queue Fronta("Fronta");                 // místo „fronta"
Facility Linka("Linka obsluhy", &Fronta);  // místo „Linka obsluhy" (1 značka),
 ↪  čeká se ve „frontě"
Histogram Doba("Doba obslouženého zákazníka [min]", 0, 10, 20);
int obslouzeno = 0, vyhozeno = 0, zavreni = 0;
Event *prichody;                        // značka v místě „příchody zákazníků"
```

```
class Hodina : public Event {          // časovaný přechod 1 h na „frontě": odpočet
↪  jednoho zákazníka
  Process *zakaznik;
 public:
  Hodina(Process *z);
  void Behavior() override;
};

class Zakaznik : public Process {      // transakce = značka, která prochází sítí
 public:
  Hodina *hodina = nullptr;
  void Behavior() override {
    double prichod = Time;
    hodina = new Hodina(this);         // značka ve „frontě": vlastní odpočet 1 h
    Seize(Linka);                      // „fronta" + obsazení linky
    delete hodina;                     // obsazení vzalo značku z „fronty", odpočet
     ↪  zaniká
    hodina = nullptr;
    Wait(Exponential(15));             // časovaný přechod exp(15 min) ...
    Release(Linka);                    // ... který vrací značku linky
    obslouzeno++;
    Doba(Time - prichod);
  }                                   // konec Behavior = místo „odchází" + přechod
   ↪  „odchod"
};

class Zavreno : public Event {         // místo „zavřeno" + časovaný přechod 3 h
  void Behavior() override {
    prichody->Activate(Time + Exponential(10));  // značka zpět do „příchody
↪  zákazníků" (nový odpočet)
  }
};

Hodina::Hodina(Process *z) : zakaznik(z) { Activate(Time + 60); }

void Hodina::Behavior() {              // zákazník čekal hodinu → „čekal 1 h"
  zavreni++;
  ((Zakaznik *)zakaznik)->hodina = nullptr;  // tenhle odpočet právě doběhl
  zakaznik->Cancel();                  // vyjme ho z fronty; odchází přes „zavření
↪  salonu"
  vyhozeno++;
  while (!Fronta.Empty()) {           // „vyprázdnění" (p=1, smyčka s „čekal 1 h"):
    Zakaznik *z = (Zakaznik *)Fronta.GetFirst();
    delete z->hodina;                  // jeho odpočet zaniká spolu se značkou
    z->Cancel();                       // každý čekající odchází přes „odchází"
    vyhozeno++;
  }
  prichody->Passivate();               // „zavření salonu": odpočet exp(10 min)
↪  zaniká
  (new Zavreno)->Activate(Time + 180); // značka do „zavřeno", za 3 h zpět
}

class Generator : public Event {       // místo „příchody zákazníků" ve smyčce s
↪  exp(10 min)
  void Behavior() override {
    (new Zakaznik)->Activate();        // nová značka v místě „fronta"
    Activate(Time + Exponential(10));  // další příchod
  }
};
```

```
int main() {
  Init(0, 60 * 1000);                // modelový čas v minutách: 1000 hodin provozu
  prichody = new Generator;
  prichody->Activate();
  Run();
  Linka.Output();
  Fronta.Output();
  Doba.Output();
  Print("Obslouženo: %d, odešlo z fronty: %d, zavření salonu: %d\n", obslouzeno,
 ↪  vyhozeno, zavreni);
}
```

Petriho síť	SIMLIB
příchody zákazníků ⇄ `exp(10 min)`	událost `Generator` (ukazatel `prichody`), která se znovu plánuje za `Exponential(10)`
značka (zákazník)	proces `Zakaznik`
místo *fronta*	`Queue Fronta`, vstupní fronta zařízení `Linka`
místo *Linka obsluhy* (1 značka) + obsazení	`Facility Linka`, `Seize(Linka)`
stříhání → `exp(15 min)` se sloučeným uvolněním	`Wait(Exponential(15));` `Release(Linka);`
časovaný přechod `1 h` na *frontě* (odpočet každého čekajícího)	událost `Hodina`, kterou si každý zákazník naplánuje před `Seize` a po něm zruší (`delete`)
čekal 1 h + smyčka `p=1` *vyprázdnění*	`zakaznik->Cancel()` a cyklus `while (!Fronta.Empty())`, který ruší čekající i jejich odpočty
zavření salonu (bere značku příchodů)	`prichody->Passivate()`
zavřeno → `3 h` → zpět *příchody zákazníků*	událost `Zavreno` za 180 min: `prichody->Activate(…)`
odchází → přechod *odchod*	konec metody `Behavior()` zákazníka nebo `Cancel()` čekajícího zákazníka

Kontrola simulací. Petriho síť: linka se vždy vrátí, platí *příchody zákazníků* + *zavřeno* = 1, v *čekal 1 h* je nejvýš jeden zákazník a fronta zůstává omezená (nejvýš 16), i když je obsluha pomalejší než příchody (`make check`). SIMLIB model: v jednom běhu (1000 h) se salon zavřel 166krát, takže byl zavřený 166 × 3 h = 498 h, tedy zhruba polovinu doby. Za zbylých 502 h přišlo asi 3000 zákazníků (502 h × 6 za hodinu): 1848 jich bylo obslouženo a 1141 odešlo z fronty při zavření. Linka byla vytížená na 47 %, nejdelší fronta měla 14 zákazníků a nikdo ve frontě nečekal déle než 60 minut (`make simlib`).

6 Původ a zdroje

Původ: půlsemestrální písemka 2025/26, varianta A, 10 bodů (bodování podle ostatních let).

Původní znění (rekonstrukce): Zadání z roku 2025/26 v materiálech chybí, existuje jen Hrubého řešení. Text níže je odvozený z jeho sítě a poznámek („čekající zákazníci

odchází ze salonu“, „deaktivace procesu příchodů“, „pokud kdokoli ve frontě čeká ≥ 1 h, pak …“). Čísla jsou převzatá z řešení.

> *Do psího salonu přichází zákazníci v intervalech daných exponenciálním rozložením se středem 10 minut. Salon má jednu linku obsluhy, obsluha (stříhání) trvá dobu danou exponenciálním rozložením se středem 15 minut. Pokud některý zákazník čeká ve frontě alespoň 1 hodinu, všichni čekající zákazníci salon opustí a salon na 3 hodiny přestane přijímat nové zákazníky. V modelu jednoznačně ukažte (přechodem) moment, kdy transakce opouští systém.*

Zdroje:

- `Fitcheats/Půlsemestrálky/Hrubý - Řešení 2025-26.pdf`, str. 1

Existující řešení:

- **Hrubý (str. 1):** síť výše vychází z jeho řešení: stejné přechody a priority. Generátor kreslí také jako místo ve smyčce s `exp(10min)`. U přechodu `1 h` má poznámku „Trvá!“ (odpočet běží, dokud je fronta neprázdná) a sám přiznává abstrakci: „pokud kdokoli ve frontě čeká ≥ 1 h“ modeluje přibližně; jako alternativu navrhuje dát každé transakci vlastní timeout. Rozdíl: Hrubý má místo „vyhlížení“ s jednou značkou, které dovolí jen jeden odpočet `1 h` (abstrakce). Tady je použitá jeho přesnější alternativa: `1 h` čte jen z *fronty*, takže má každý čekající vlastní odpočet, a místo „vyhlížení“ odpadá.